PROJECT-BRIEF: CueGate · Deepak Kumaran T S · INFO 7375 · Fall 2026 · 2026-10-14

1. What is it? A subtitle checker with a repair loop: CueGate will find every subtitle a viewer cannot read in time, fix timing first, shorten text only where timing cannot fix it, and refuse to ship any line it cannot prove fits. It will be a Python package (`cuegate`), a command-line tool, a local web app, and an evaluation report on Creative Commons open-movie subtitles.

2. Why would anyone care? Streaming platforms publish numeric subtitle rules (reading speed, gaps between events, timing at shot changes); delivery guides report that files breaking them are rejected in QC. I looked at Subtitle Edit, Editsub and EZTitles: the first two offer AI shortening of one line on request, and EZTitles runs automated QC that flags reading-speed violations; there, a person triggers shortening line by line. CueGate will save a QC lead that pass with a fail-closed gate over the whole file, and will publish how often its shortened lines changed meaning, measured against a hand-labelled audit. I found no published figure like that for these tools (re-checked 2026-10-14, evidence/day-one/).

3. Who opens it, and what do they do? Target role: Applied AI Engineer / AI Engineer. Reader: a subtitle QC lead at a localization vendor, and a hiring manager who runs the evaluation. In the first sixty seconds they will drop in a failing subtitle file plus the film or a cut list, and get a table of every broken rule, every repaired line and how, and every line CueGate will refuse to change, with the reason.

4. What does it prove about me? Prompt contracts (condense call with exact character budget, must-keep list, JSON schema); verification (deterministic rule engine and meaning invariants that will decide acceptance, not the model); agent loops (bounded repair loop, failure feedback, escalation); evaluation (four baselines, stress generator, blind audit); source grounding (every rule and violation cites its guideline section and read date). Evidence will be the repo's tests, evaluation report and audit labels.

5. In, out, first. In: English SRT/WebVTT, shot detection or a cut list, timing, line-break, resegment and condense repairs, meaning guard, evaluation, web app. Out, deliberately: translation. First: `cuegate check` on one open film's English subtitles; proof: its counts will match a hand-checked fixture in CI.

6. What could sink it? Shot detection: fades and fast motion can create phantom cuts that misfire the timing rules. Response: an adaptive detector with a minimum scene length, a cut-list override, and detector errors reported apart from repair results. Deeper limitation: a line can pass every rule and still change meaning, so the meaning guard's false-pass rate will be measured and published, not assumed.

7. Why this project? A mix. Existing skills: evaluation harnesses and prompt-regression testing (from my Twin AI co-op; not public). Public evidence of verification work: the timeline gate in my Reallocation Engine recipe for this course, https://github.com/nikbearbrown/the-reallocation-engine/pull/25 (commit edbac80), built with Claude Code as co-author. For evaluation specifically, the evidence will be this project's own test suite. Other existing skills: Python services and CI. New to me: frame-accurate timing, shot detection, subtitle standards, NLI-based meaning checks, and dynamic-programming line breaking.

Exists today

- The planning README and planning PDF for CueGate (both 2026-10-14).
- A rule profile, written as a YAML block in section 8 of the plan, with values checked against Netflix's guideline pages on 2026-10-14, except whether spaces count, which those pages do not state.
- Dated evidence: the Claude exchange shown in the film, and the day-one source checks (evidence/day-one/), both 2026-10-14.

Will exist

- The `cuegate` package, CLI and web app.
- Rule engine, timing solver, line breaker, resegmenter, condenser, meaning guard, bounded loop.
- Licensed open-movie corpus, stress generator, baselines, blind audit, evaluation report.